

Relatório - Comparação de Algoritmos de Ordenação

Resumo

- Algoritmos testados: **Bubble Sort**, **Insertion Sort**, **Quick Sort** (pivô = último elemento).
- Conjuntos: 9 arquivos em `conjuntosDeDados/` (aleatório, crescente, decrescente — tamanhos 100, 1.000, 10.000).
- Medição: `System.nanoTime()`. Resultados completos em `results.csv`.

Tabela de resultados (tempos em nanosegundos)

Arquivo	Bubble	Insertion	Quick
aleatorio_100.csv	149625	41458	24291
aleatorio_1000.csv	3636041	1300958	173875
aleatorio_10000.csv	58131917	18849792	754584
crescente_100.csv	667	14584	15375
crescente_1000.csv	708	24375	351958
crescente_10000.csv	3500	40333	25263292
decrescente_100.csv	24083	3084	22667
decrescente_1000.csv	874875	206292	225583
decrescente_10000.csv	87532291	9010625	20201667

Breve análise

- Dados aleatórios:
 - Quick Sort foi claramente o mais rápido para os três tamanhos (por margem crescente com o tamanho).
 - Insertion ficou entre Quick e Bubble; Bubble é o mais lento, refletindo $O(n^2)$ vs $O(n \log n)$.
- Dados já ordenados (crescente):
 - Bubble teve tempos muito baixos graças à otimização de "early exit" (verificação de trocas). Para 1000 e 10000 o tempo permanece muito pequeno.
 - Insertion também é eficiente em entradas quase-ordenadas (complexidade próxima de $O(n)$ para entradas quase ordenadas).
 - Quick Sort (com pivô = último) apresentou comportamento ruim no `crescente_10000.csv` (tempo muito alto). Isso ocorre porque escolher sempre o último elemento como pivô em uma entrada já ordenada gera partições muito desbalanceadas, aproximando Quick Sort de $O(n^2)$.
- Dados decrescentes (ordenação inversa):

- Bubble e Insertion apresentam alto custo para tamanhos grandes (n^2), Bubble especialmente custoso no `decrecente_10000.csv`.
- Quick Sort também é afetado pelo pivô fixo; o tempo para `decrecente_10000.csv` é grande (mas menor que Bubble aqui), já que a partição também tende a ser desbalanceada.

Conclusões e recomendações

- Para entradas aleatórias e gerais, Quick Sort é a melhor escolha (complexidade média $O(n \log n)$).
- Para entradas já quase-ordenadas, Insertion Sort e Bubble (com early exit) podem ter desempenho muito bom; porém Insertion é geralmente preferível por ser mais consistente e ter menor overhead.
- O Quick Sort implementado aqui (pivô = último elemento) tem pior caso para entradas ordenadas/anti-ordenadas; recomenda-se usar:
 - pivô aleatório (randomized pivot) ou
 - estratégia median-of-three para evitar comportamento quadrático em entradas adversas.